

## PRACTICAL\_WEEK-10

**Task 1:** Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % echo "Practical 10 inode test file" > inode_test.txt
amrutha@Amruthas-MacBook-Air OSSP % cat inode_test.txt
Practical 10 inode test file
amrutha@Amruthas-MacBook-Air OSSP %
```

**Fig 1:** Creation of the sample file for inode and link analysis

```
amrutha@Amruthas-MacBook-Air OSSP % ls -li inode_test.txt
15669285 inode_test.txt
amrutha@Amruthas-MacBook-Air OSSP % stat inode_test.txt
16777233 15669285 -rw-r--r-- 1 amrutha staff 0 29 "Sep 23 10:54:03 2026" "Sep 23 10:54:02 2026" "Sep 23 10:54:02 2026" "
Sep 23 10:54:02 2026" 4096 8 0 inode_test.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

**Fig 2:** Display of inode number and file metadata using `ls -li` and `stat` commands

```
amrutha@Amruthas-MacBook-Air OSSP % ln inode_test.txt inode_hardlink.txt
amrutha@Amruthas-MacBook-Air OSSP % ls -li inode_test.txt inode_hardlink.txt
15669285 inode_hardlink.txt      15669285 inode_test.txt
amrutha@Amruthas-MacBook-Air OSSP % stat inode_hardlink.txt
16777233 15669285 -rw-r--r-- 2 amrutha staff 0 29 "Sep 23 10:58:01 2026" "Sep 23 10:54:02 2026" "Sep 23 10:57:59 2026" "Sep 23 10:54:02 2026" 4096 8 0 inode_hardlink.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

**Fig 3:** Creation of a hard link and verification that both files share the same inode

```
16777233 15669285 -rw-r--r-- 2 amrutha staff 0 29 "Sep 23 10:58:01 2026" "Sep 23 10:54:02 2026" "Sep 23 10:57:59 2026" "Sep 23 10:54:02 2026" 4096 8 0 inode_hardlink.txt
amrutha@Amruthas-MacBook-Air OSSP % ln -s inode_test.txt inode_symlink.txt
amrutha@Amruthas-MacBook-Air OSSP % ls -li inode_test.txt inode_symlink.txt
15671620 inode_symlink.txt      15669285 inode_test.txt
amrutha@Amruthas-MacBook-Air OSSP % ls -li inode_symlink.txt
lrwxr-xr-x 1 amrutha staff 14 Sep 23 11:02 inode_symlink.txt -> inode_test.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

**Fig 4:** Creation of a symbolic link and verification that it has a different inode from the original file

```
amrutha@Amruthas-MacBook-Air OSSP % find . -inum 15669285
[amrutha@Amruthas-MacBook-Air OSSP % ls -li inode_test.txt
15669285 inode_test.txt
[amrutha@Amruthas-MacBook-Air OSSP % find . -inum 15669285
./inode_test.txt
./inode_hardlink.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

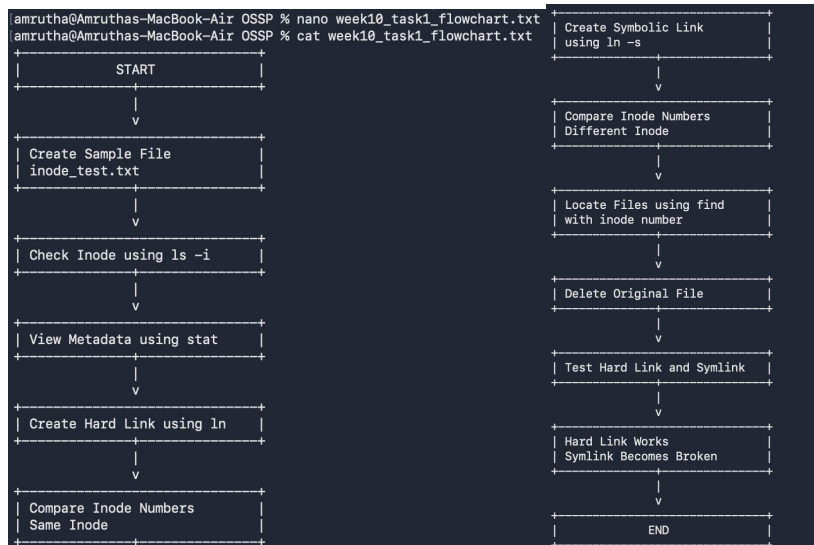
**Fig 5:** Locating files that share the same inode using the `find` command

```

7 inode_hardlink.txt
[amrutha@Amruthas-MacBook-Air OSSP % rm inode_test.txt
[amrutha@Amruthas-MacBook-Air OSSP % cat inode_hardlink.txt
Practical 10 inode test file
[amrutha@Amruthas-MacBook-Air OSSP % cat inode_symlink.txt
cat: inode_symlink.txt: No such file or directory
[amrutha@Amruthas-MacBook-Air OSSP % ls -l inode_symlink.txt
lrwxr-xr-x 1 amrutha staff 14 Sep 23 11:02 inode_symlink.txt -> inode_test.txt
[amrutha@Amruthas-MacBook-Air OSSP % █

```

**Fig 6:** Behavior of hard and symbolic links after deletion of the original file



**Fig 7:** Flowchart for inode analysis and comparison of hard links and symbolic links

## Observation:

The original file and hard link shared the same inode number, while the symbolic link had a different inode. After deleting the original file, the hard link remained accessible, whereas the symbolic link became broken.

## Result:

Inode structures were analyzed successfully using `ls -i`, `stat`, and `find`. Hard links and symbolic links were created and compared, demonstrating their different effects on inode allocation and file accessibility.

**Task 2:** Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional `read()/write()` operations in terms of efficiency and implementation complexity.

```
UW PICO 5.09

}

char *mapped = mmap(NULL,
                    size,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED,
                    fd,
                    0);

if (mapped == MAP_FAILED)
{
    perror("mmap failed");
    close(fd);
    return 1;
}

strcpy(mapped, message);

printf("Data written using mmap():\n%s\n", mapped);

if (msync(mapped, size, MS_SYNC) == -1)
{
    perror("msync failed");
}

printf("\nData read from mapped memory:\n%s\n", mapped);

munmap(mapped, size);
close(fd);

printf("\nMemory mapping completed successfully.\n");

return 0;
}
```

**Fig 8:** C program implementing file reading and writing using `mmap()`

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/week10_task2.c -o bin/week10_task2
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/week10_task2
Data written using mmap():
Operating Systems mmap file I/O demonstration.

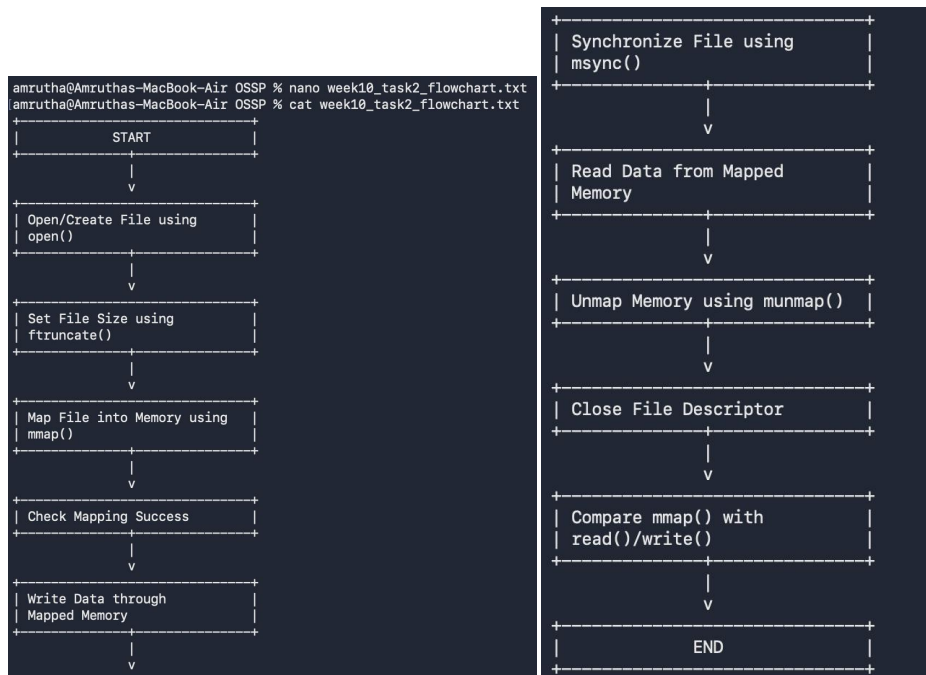
Data read from mapped memory:
Operating Systems mmap file I/O demonstration.

Memory mapping completed successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

**Fig 9:** Execution output demonstrating file reading and writing using `mmap()`

```
Memory mapping completed successfully.
[amrutha@Amruthas-MacBook-Air OSSP % cat mmap_file.txt
Operating Systems mmap file I/O demonstration.%
amrutha@Amruthas-MacBook-Air OSSP %
```

**Fig 10:** Verification of data written to the file through memory-mapped I/O



**Fig 11:** Flowchart for file reading and writing using memory-mapped I/O with `mmap()`

## Observation:

The file was successfully mapped into virtual memory using `mmap()`. Data was written and read through the mapped memory region and synchronized with the file using `msync()`. Memory-mapped I/O provided direct memory-style access to file contents.

## Result:

File reading and writing using `mmap()` was implemented successfully. Memory-mapped I/O was studied and compared with traditional `read()` and `write()` operations in terms of implementation and data-access method.